Dataset	Total Steps	% Correct	% Incorrect
Synthetic (Boolean)	126,624	99.8%	0.2%
Synthetic (Arithmetic)	155,434	98.8%	1.2%
GSM8K	8,737	93.4%	6.6%

Table 12 Final statistics of our curated datasets, showing the number of reasoning steps and the distribution of correct/incorrect labels after our full annotation and filtering process.

Dataset	Difficulty (Operators)	Final Answer Accuracy
Synthetic (Boolean)	n=3	98.4%
	n=5	93.27%
	n=7	89.4%
	n=10	78.43%
Synthetic (Arithmetic)	n=3	94.83%
	n=5	86.8%
	n=7	73.07%
	n=10	52.8%
GSM8K	-	75.82%

Table 13 End-to-end task accuracy of our base model (Llama 3.1 8B Instruct). For the synthetic datasets, we provide a fine-grained breakdown by difficulty, controlled by the number of operators (n).

B Transcoder Training Details

Our methodology relies on high-fidelity, sparsely activating transcoders to create an interpretable model. To this end, we trained a TopK-Transcoder for each target MLP module in the Llama 3.1 8B Instruct model. Our training protocol is designed for robustness and follows several best practices established in recent literature.

The transcoders were trained on a high-quality, 10B token subset of the RedPajama-V2 dataset (Weber et al., 2024). We pre-processed the entire training corpus by concatenating and chunking all passages into a uniform length, and we explicitly discarded all beginning-of-sequence (BOS) tokens[7], which we found to be detrimental to stable transcoder training. The transcoder architecture consists of a simple autoencoder with a single hidden layer and a ReLU activation. For each MLP layer in the base model, the transcoder is trained to take the residual stream before the MLP block as input and reconstruct the residual stream after the MLP's computation. The input dimension matches the Llama 3.1 8B's MLP hidden dimension (4096), and the latent feature dimension was set to an overcomplete basis of 131,072. We enforced sparsity structurally using a TopK mechanism, preserving only the $k = 128$ largest feature activations in the forward pass.

We followed several established training techniques to improve feature quality and avoid common pitfalls (Gao et al., 2025; Yang et al., 2025). The decoder weights were normalized to have unit norm, and we did not tie the encoder and decoder weights. To prevent feature collapse, we implemented a dead neuron revival mechanism: if a feature neuron had not activated in 10 million tokens, its activation was forced with an auxiliary loss (coefficient of 1/32).

The transcoders were trained for 4 epochs using the AdamW optimizer. The learning rate was set to 7e-5 with a warmup ratio of 0.5. Training was conducted on 4 nodes, each with 8 Nvidia H200 GPUs, using a total batch size of 4,096. This was achieved with a per-device batch size of 32 and gradient accumulation steps. We found that the training loss generally saturated after approximately 4,000 steps, indicating efficient convergence. We show the training loss on selected layers in Figure 5.

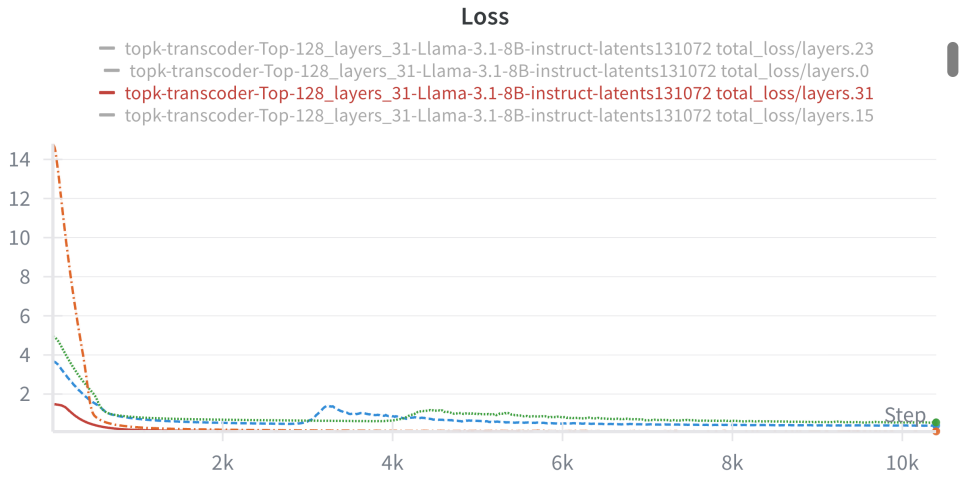

Figure 5 Transcoder Training Loss Curves. The x-axis represents training steps. In all cases, the loss converges efficiently, generally saturating after approximately 4,000 steps.

B.1 Impact of Training Transcoders on Instruction-Tuning Data

Since our base LLM used is an instruct model, a natural hypothesis is that transcoders fine-tuned on instruction-following data might learn features more relevant to CoT reasoning, thereby improving verification performance. To test this, we trained an version of our transcoders with instruction-tuning (IT) data. Starting from our pre-trained base transcoders, we continued fine-tuning for 1 epoch on the LMSYS-Chat-1M dataset (Zheng et al., 2024), using the same hyperparameters

[7]BOS tokens are retained when generating activations but their activations are removed afterward for training the transcoders.

Transcoder Training	Synthetic (Boolean)			Synthetic (Arithmetic)			GSM8K		
	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓
Base	75.87	0.97	79.17	92.47	28.92	37.09	70.17	14.3	79.61
+ IT Data	76.04	1.20	66.82	91.39	28.44	38.47	72.01	15.40	83.27

Table 14 Performance comparison of CRV with Base transcoders vs. transcoders further trained on Instruction-Tuning (IT) data. Arrows indicate preferred direction (↑ higher is better, ↓ lower is better).

as for the base transcoder training. Following the methodology of Lieberum et al. (2024), we prepended and appended the Llama 3.1 8B Instruct model's IT prefixes to the user queries and model responses respectively.

However, as shown in Table 14, this additional training on IT data did not yield a consistent or meaningful improvement in verification performance on our tasks. This finding is consistent with recent work by Kissane et al. (2024), who found that SAEs trained on base model activations can also faithfully reconstruct the activations of derived IT models. While a deeper mechanistic investigation into how instruction-tuning affects the underlying feature space is a promising direction, we leave this for future work. For our main experiments, we therefore use the more general base transcoders.

B.2 Attribution Graph Computation

Implementation Details. We use the implementation from Hanna et al. (2025) to compute attribution graphs. The primary hyperparameters were set as follows: a maximum of 4096 feature nodes, attribution traced from a maximum of 10 logit nodes (selected by a cumulative probability threshold of 0.95), and a batch size of 16 for backward passes. All other parameters follow the repository defaults.

Attribution Position	Synthetic (Boolean)			Synthetic (Arithmetic)			GSM8K		
	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓
Before	68.66	1.80	77.44	85.95	12.05	47.89	70.32	16.19	85.29
After	75.87	0.97	79.17	92.47	28.92	37.09	70.17	14.3	79.61

Table 15 Performance comparison of CRV using different token positions for attribution graph computation. The "After" setting computes the graph at the final token of the current step, while "Before" uses the final token of the previous step. Arrows indicate preferred direction (↑ higher is better, ↓ lower is better).

Ablation on Attribution Position. The attribution graph is computed with respect to a specific token position. The choice of this position is a critical methodological decision, as it determines which computational moment we analyze. We investigate two hypotheses: analyzing the state before a step is generated (the "pre-computation" trace) versus the state after it is complete (the "post-computation" trace). To test this, we compare two settings: (1) Before: computing the graph at the position of the final token of the previous reasoning step. For the first step of the CoT, this corresponds to the final token of the input question. (2) After: computing the graph at the final token of the current reasoning step, which is the default setting for our main experiments.

The results, presented in Table 15, show a clear and consistent advantage for the "After" setting across nearly all metrics and domains. We hypothesize that this is because the structural signatures of a flawed computation are most fully consolidated in the final token's representation after the step has been fully executed. The pre-computation state may contain signals of intent or planning, but the post-computation state contains the definitive trace of the executed algorithm, including the evidence of its failure. Based on these results, all experiments in the main body of the paper use the "After" (current step) position.